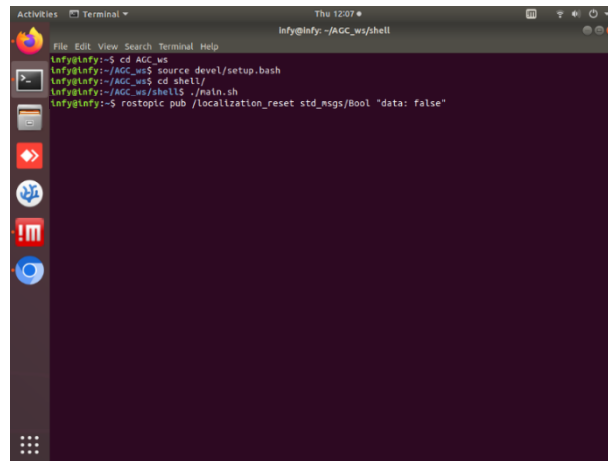


Execution Steps

Step-1:

The system was started by turning on the vehicle and initializing the autonomous environment. All the necessary autonomous driving modules were started using predefined command-line scripts.



```
infy@infy:~/AGC_ws$ cd AGC_ws
infy@infy:~/AGC_ws$ source devel/setup.bash
infy@infy:~/AGC_ws$ cd shell/
infy@infy:~/AGC_ws/shell$ ./main.sh
infy@infy:~/AGC_ws/shell$ rostopic pub /Localization_reset std_msgs/Bool "data: false"
```

Fig 1.1 Commands to launch all models & localize the vehicle

The autonomous driving modules included vehicle control nodes, localization module, perception module, and communication interfaces necessary for the buggy to run in autonomous mode. The start process was done using a number of configured scripts.

Step-2:

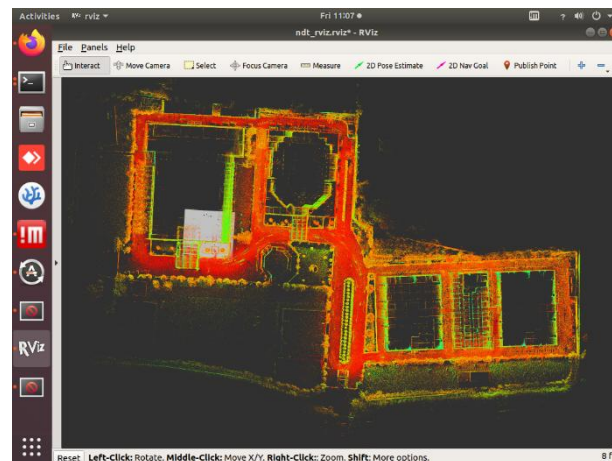


Fig 1.2 Complete College map

After successfully launching all autonomous modules, a ROS visualization (Rviz) platform is launched which shows complete recorded map with vehicle Cost map, where vehicle is placed in some random position.

Step-3:

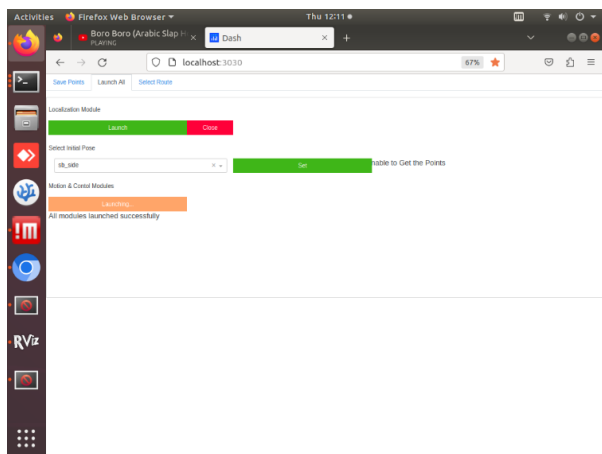


Fig 1.3 Selected the localize point as sd_side

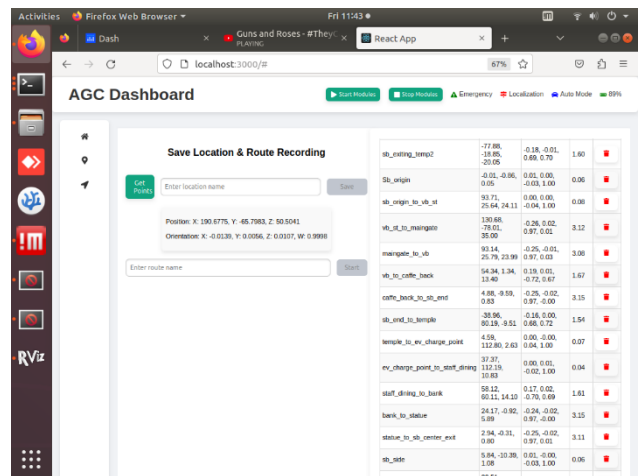


Fig 1.4 Save Points

We opened a dashboard in a browser, using the dashboard interface the vehicle has to be localization to the origin position (or) some save point position.

Step-4:

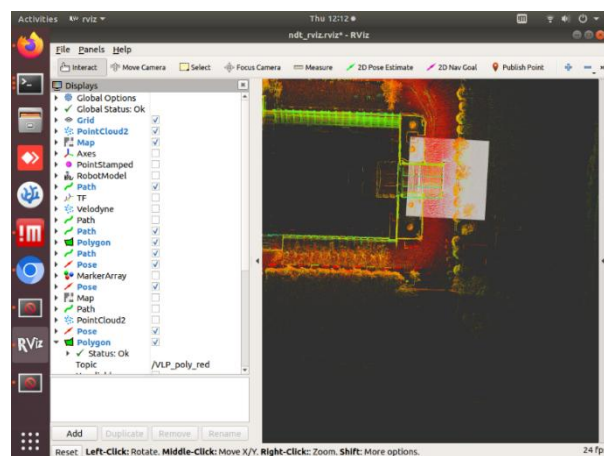


Fig 1.5 Cost map localizing to current position

The localization module was used to determine the current position of the vehicle in the mapped environment. Successful localization was important to ensure that the vehicle takes the correct route to the destination.

Step-5:

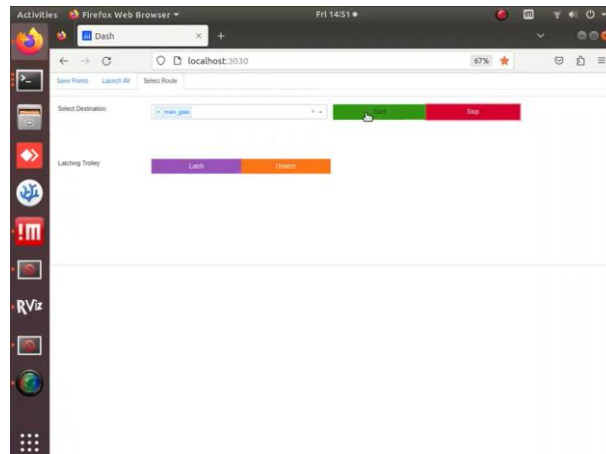


Fig 1.6 Vehicle is set to the destination position

After successful localization, a destination point was assigned using the dashboard interface. The vehicle's navigation generated a path from the source to the destination point, and the buggy started moving autonomously along the path.

Step-6:

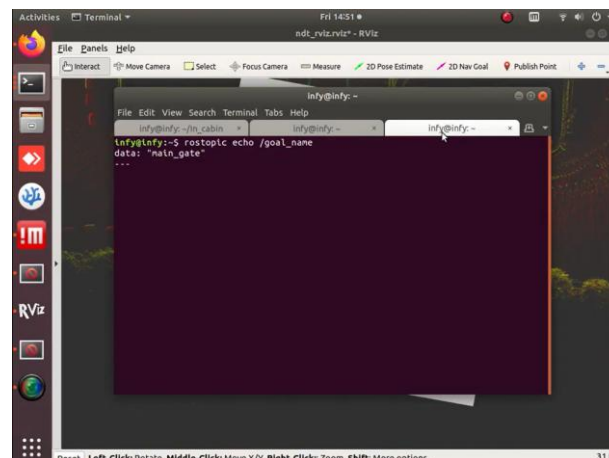


Fig 1.7 Vehicle assigned to destination

Where in the goal name is also set to the main_gate which is taken as destination.

Step-7:

The system was constantly checking the posture of the passengers, blinking of the eyes, and heart rate. A multi-validation logic was used to come up with the final decision:

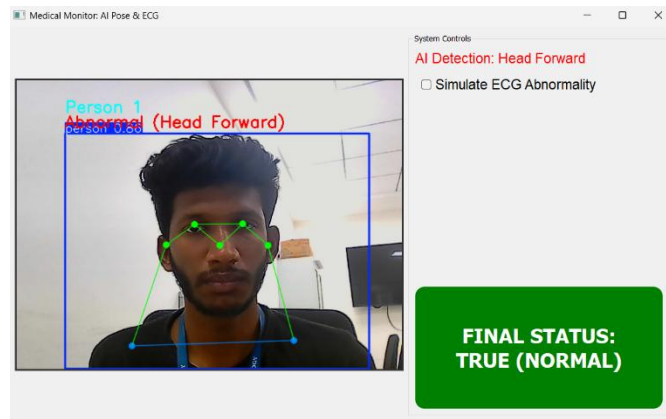


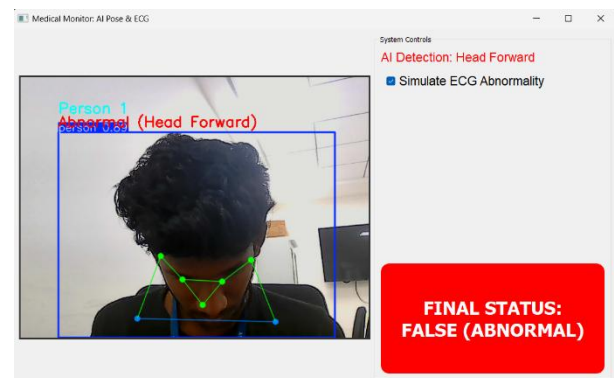
Fig 1.7 Only posture shown as abnormal, final condition shown as True

If Posture is in abnormal condition, it check the heart rate if it is normal then, it is shown as true.

Step-8:



(a)



(b)

Fig 1.8 If Both posture and heart rate shown as abnormal then, final condition shown as False

The Fig 1.8 (a) tells if ((Posture abnormality == True) and (Heart rate abnormality == True)). Provided that both of these conditions were met, the system produced a Boolean value of False as shown in Fig 1.8 (b), which means an emergency condition.

Step-9:

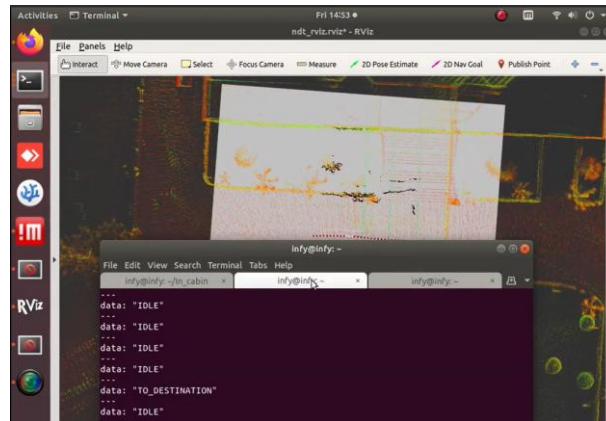


Fig 1.9 Vehicle moving from Destination to IDLE state

When the vehicle got the emergency signal, it slowed down safely and goes from the destination to the idle state.

Step-10:

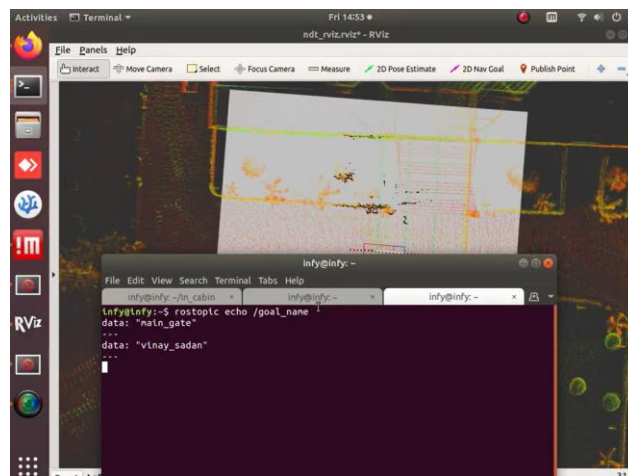


Fig 1.10 Vehicle is rerouted to the health care point

After going to the idle state, the vehicle is rerouted to the nearest hospital. In our case we choose the Vinay Sadan block as health care point. So, the vehicle is rerouted to that point when the abnormality is detected. This process continuously monitors passenger health and behaviour using a deep learning model and integrates sensors with the HPC to identify potential medical abnormalities. This ensures timely intervention and enhances passenger safety.

Codes

a) Train the model

- 1) `!pip install ultralytics`
- 2) `from ultralytics import YOLO`

`print("YOLOv8 installed successfully")`
- 3) `from google.colab import drive`

`drive.mount('/content/drive')`

`yaml_content_for_data_file = f"""`
`path: /content/drive/MyDrive/dataset`
`train: images/train`
`val: images/val`
`test: images/test`
`nc: 2`
`names: ['person_abnormal', 'person_normal']`
`"""`
- 4) `data_yaml_path = '/content/drive/MyDrive/dataset/data.yaml'`

`with open(data_yaml_path, 'w') as f:`

`f.write(yaml_content_for_data_file)`

`print(f"Created {data_yaml_path} with the following content :\n{yaml_content_to_file}")`
- 5) `from ultralytics import YOLO`

`model = YOLO("YOLOv8n.pt")`

`model.train(`

`data="/content/drive/MyDrive/dataset/data.yaml",`

`epochs=10,`

`imgsz=640,`

`batch=5`

`)`

b) Abnormality of the passenger detection and sending the signal to the vehicle

```
import sys

import cv2

import numpy as np

import socket

from ultralytics import YOLO

from PyQt5.QtWidgets import (QApplication, QMainWindow, QWidget, QLabel,
                              QVBoxLayout, QHBoxLayout, QCheckBox, QGroupBox)

from PyQt5.QtGui import QPixmap, QImage, QFont

from PyQt5.QtCore import pyqtSignal, pyqtSlot, Qt, QThread

# ===== UDP CONFIGURATION =====

UDP_IP = "192.168.0.4"

UDP_PORT = 5005

# ===== BACKGROUND VIDEO THREAD =====

class VideoThread(QThread):

    change_pixmap_signal = pyqtSignal(np.ndarray)

    posture_signal = pyqtSignal(bool, str)

    def __init__(self):

        super().__init__()

        self._run_flag = True

        self.pose_model = YOLO("best.pt")

    def run(self):

        cap = cv2.VideoCapture(0)

        while self._run_flag:

            ret, frame = cap.read()

            if not ret:
```

```

        break

    results = self.pose_model.predict(frame, stream=True, classes=[0],
verbose=False)

    is_posture_abnormal = False

    reason = "Normal"

    person_count = 0

    for r in results:

        frame = r.plot()

        if r.keypoints is not None and len(r.keypoints.xy) > 0:

            kpts = r.keypoints.xy.cpu().numpy()

            for i, kpt in enumerate(kpts):

                person_count += 1

                code, reason = self.classify_posture(kpt)

                x1, y1, x2, y2 = map(int, r.bboxes.xyxy[i])

                cv2.putText(

                    frame, f"Person {person_count}",

                    (x1, y1 - 35),

                    cv2.FONT_HERSHEY_SIMPLEX, 0.9,

                    (255, 255, 0), 2

                )

            if code != 0:

                label_map = {

                    1: "Abnormal (Head Forward)",

                    2: "Abnormal (Head Backward)",

                    3: "Abnormal (Lean Left)",

                    4: "Abnormal (Lean Right)"

                }

                cv2.putText(

```

```

        frame, label_map[code],
        (x1, y1 - 10),
        cv2.FONT_HERSHEY_SIMPLEX, 0.9,
        (0, 0, 255), 2
    )

    is_posture_abnormal = True

    print(f"Posture Code: {code}, Reason: {reason}")

    self.change_pixmap_signal.emit(frame)

    self.posture_signal.emit(is_posture_abnormal, reason)

    if not self._run_flag:
        break

    cap.release()

def stop(self):
    self._run_flag = False
    self.wait()

def classify_posture(self, k):
    try:
        head_tilt = k[0][1] - (k[1][1] + k[2][1]) / 2
        shoulder_diff = k[5][1] - k[6][1]

        if head_tilt > 25: return 1, "Head Forward"
        if head_tilt < 8: return 2, "Head Backward"
        if shoulder_diff > 10: return 3, "Lean Left"
        if shoulder_diff < -25: return 4, "Lean Right"

        return 0, "Normal"

    except Exception:
        return 0, "Normal"

# ===== MAIN GUI WINDOW =====

class MainWindow(QMainWindow):

```

```

def __init__(self):
    super().__init__()
    self.setWindowTitle("Medical Monitor: AI Pose & ECG")
    self.resize(1100, 650)
    self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    self.last_sent = None
    self.posture_bad = False
    self.ecg_bad = False
    self.locked_false = False # latch flag
    self.init_ui()
    self.thread = VideoThread()
    self.thread.change_pixmap_signal.connect(self.update_image)
    self.thread.posture_signal.connect(self.update_posture)
    self.thread.start()

def init_ui(self):
    central_widget = QWidget()
    self.setCentralWidget(central_widget)
    main_layout = QHBoxLayout(central_widget)
    self.video_label = QLabel()
    self.video_label.setFixedSize(640, 480)
    self.video_label.setScaledContents(True)
    self.video_label.setStyleSheet("border: 3px solid #333; background: black;")
    main_layout.addWidget(self.video_label)
    controls_group = QGroupBox("System Controls")
    controls_layout = QVBoxLayout()
    self.lbl_posture = QLabel("Posture: Initializing...")
    self.lbl_posture.setFont(QFont("Arial", 14))
    self.lbl_posture.setStyleSheet("color: blue; padding: 10px;")

```

```

self.chk_ecg = QCheckBox("Simulate ECG Abnormality")
self.chk_ecg.setFont(QFont("Arial", 14))
self.chk_ecg.setStyleSheet("padding: 10px;")
self.chk_ecg.toggled.connect(self.update_ecg)
self.lbl_final = QLabel("FINAL STATUS:\nTRUE (NORMAL)")
self.lbl_final.setAlignment(Qt.AlignCenter)
self.lbl_final.setMinimumHeight(200)
self.lbl_final.setStyleSheet("background: green; color: white; font-size: 20pt; font-
weight: bold; border-radius: 15px;")
controls_layout.addWidget(self.lbl_posture)
controls_layout.addWidget(self.chk_ecg)
controls_layout.addStretch()
controls_layout.addWidget(self.lbl_final)
controls_group.setLayout(controls_layout)
main_layout.addWidget(controls_group)
@pyqtSlot(np.ndarray)
def update_image(self, img):
    rgb_img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    h, w, c = rgb_img.shape
    qimg = QImage(rgb_img.data, w, h, w * c, QImage.Format_RGB888)
    self.video_label.setPixmap(QPixmap.fromImage(qimg))
@pyqtSlot(bool, str)
def update_posture(self, is_bad, reason):
    self.posture_bad = is_bad
    self.lbl_posture.setText(f"AI Detection: {reason}")
    self.lbl_posture.setStyleSheet(f"font-size: 14pt; color: {'red' if is_bad else 'green'};")
    self.calculate_final_result()
def update_ecg(self, checked):

```

```

self.ecg_bad = checked

self.calculate_final_result()

def calculate_final_result(self):
    # Once locked_false is set, never send true again
    if self.locked_false:
        current_signal = "false"
    else:
        is_abnormal = self.posture_bad and self.ecg_bad
        current_signal = "false" if is_abnormal else "true"
    if current_signal == "false":
        self.lbl_final.setText("FINAL STATUS:\nFALSE (ABNORMAL)")
        self.lbl_final.setStyleSheet("background: red; color: white; font-size: 20pt; font-
weight: bold; border-radius: 15px;")
        self.locked_false = True # latch permanently
    else:
        self.lbl_final.setText("FINAL STATUS:\nTRUE (NORMAL)")
        self.lbl_final.setStyleSheet("background: green; color: white; font-size: 20pt; font-
weight: bold; border-radius: 15px;")
    if current_signal != self.last_sent:
        try:
            message = f"Final State: {current_signal}"
            self.sock.sendto(message.encode(), (UDP_IP, UDP_PORT))
            print(f"UDP Packet Sent: {message}")
            self.last_sent = current_signal
        except Exception as e:
            print(f"Transmission Error: {e}")

def closeEvent(self, event):
    self.thread.stop()
    self.sock.close()

```

```

        event.accept()
if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec_())

```

c) Receiving signal by the vehicle and the action performed as per the signal

```

#!/usr/bin/env python2

import socket
import rospy
import pandas as pd
import os
import sys

from std_msgs.msg import String, Bool
from nav_msgs.msg import Path
from geometry_msgs.msg import PoseStamped

# UDP Configuration
UDP_IP = "0.0.0.0"
UDP_PORT = 5005

# Pickle File Path
PICKLE_PATH = r'/home/infy/AGC_ws/intial_points.pkl'

def get_point_stamped(df, goal_name):
    """
    Retrieves the goal pose from the DataFrame based on the goal name.
    """
    if goal_name not in df['name'].values:
        return None

```

```

for ind_goal in df.index:

    if df['name'][ind_goal] == str(goal_name):

        goal = PoseStamped()

        goal.header.frame_id = 'map'

        goal.header.stamp = rospy.Time.now()

        # Extract coordinates

        pos = df['position'][ind_goal]

        orient = df['orientation'][ind_goal]

        goal.pose.position.x = pos[0]

        goal.pose.position.y = pos[1]

        goal.pose.position.z = pos[2]

        goal.pose.orientation.x = orient[0]

        goal.pose.orientation.y = orient[1]

        goal.pose.orientation.z = orient[2]

        goal.pose.orientation.w = orient[3]

        return goal

    return None

def publish_navigation_goal(pub_list, pub_name, pub_status, pub_brake, df, goal_name):
    """
    Helper function to publish all topics needed for navigation.
    """

    # 1. Release Emergency Brake

    brake_msg = Bool()

    brake_msg.data = False

    pub_brake.publish(brake_msg)

    if df is not None:

        pose = get_point_stamped(df, goal_name)

        if pose:

```

```

    # Publish Path (Required by Behavioural Planner)

    path_msg = Path()
    path_msg.header.frame_id = 'map'
    path_msg.header.stamp = rospy.Time.now()
    path_msg.poses.append(pose)
    pub_list.publish(path_msg)

    # Publish Goal Name

    goal_msg = String()
    goal_msg.data = goal_name
    pub_name.publish(goal_msg)

    # Publish Status

    status_msg = String()
    status_msg.data = "TO_DESTINATION"
    pub_status.publish(status_msg)
    rospy.loginfo("Navigation started successfully for goal: {}".format(goal_name))
    return True

else:
    rospy.logerr("Goal '{}' not found in database!".format(goal_name))
    return False

else:
    rospy.logerr("Destination database not loaded! Cannot navigate.")
    return False

def receiver():
    # Initialize ROS Node

    rospy.init_node('vision_bridge', anonymous=True)

    # Initialize Publishers

    pub_status = rospy.Publisher('/gcart_status', String, queue_size=1, latch=True)
    pub_brake = rospy.Publisher('/leg_brake', Bool, queue_size=1, latch=True)

```

```

pub_goal_name = rospy.Publisher('/goal_name', String, queue_size=1, latch=True)
pub_goal_list = rospy.Publisher('/goal_list', Path, queue_size=1, latch=True)

# Initialize UDP Socket
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind((UDP_IP, UDP_PORT))

# Load Destination Data
df = None

try:
    if os.path.exists(PICKLE_PATH):
        df = pd.read_pickle(PICKLE_PATH)
        rospy.loginfo("Loaded destinations from {}".format(PICKLE_PATH))
        rospy.loginfo("Available Goals: {}".format(list(df['name'])))
    else:
        rospy.logwarn("Could not find intial_points.pkl at {}".format(PICKLE_PATH))
except Exception as e:
    rospy.logerr("Error loading pickle file: {}".format(e))

# Track state to avoid spamming topics
last_state = None
last_target = None

rospy.loginfo("Vision Bridge Started. Listening on UDP port {}".format(UDP_PORT))

while not rospy.is_shutdown():
    try:
        # Receive Data
        data, addr = sock.recvfrom(1024)
        message = data.decode().strip()
        target_goal = None
        current_state_key = None # Used for 'last_state' tracking
        # Determine Target based on Message

```

```

if "Final State: false" in message:

    # ABNORMAL -> VINAY SADAN

    target_goal = "vinay_sadan"

    current_state_key = "FALSE"


elif "Final State: true" in message:

    # NORMAL -> SB EXITING

    target_goal = "sb_exiting_temp2"

    current_state_key = "TRUE"

# Execute Logic if state changed OR target changed

if target_goal and (current_state_key != last_state or target_goal != last_target):

    rospy.loginfo("State Change Detected: {} -> Target:
{}".format(current_state_key, target_goal))

    success = publish_navigation_goal(pub_goal_list, pub_goal_name, pub_status,
pub_brake, df, target_goal)

    if success:

        last_state = current_state_key

        last_target = target_goal

    else:

        # If failed (e.g., goal not found), allow retrying on next loop

        # by NOT updating last_state, or handle differently?

        # For now, we log error and don't update state to prevent spam if it's
permanent error,

        # but maybe we should allow retries.

        # Let's NOT update last_state so it retries if data comes again?

        # Actually, better to update partially or just log.

        pass

except socket.error as e:

    rospy.logerr("Socket Error: {}".format(e))

```

```
        rospy.sleep(1) # Prevent busy loop on socket error
    except Exception as e:
        rospy.logerr("Error: {}".format(e))
if __name__ == '__main__':
    try:
        receiver()
    except rospy.ROSInterruptException:
        pass
except Exception as e:
    print("Detailed Error: ",e)
```